Assignment 6-Team Bucur Mihnea and Bugnaru Tudor

1.Overview of Testing
Software testing is a crucial part of the Software Development Life Cycle (SDLC) and is applied at every stage to ensure the quality, functionality, security, and performance of the software. It is used both during and after development to detect bugs, verify that the system meets requirements, and prevent failures in production. Testing is useful because it increases user confidence, enhances product quality, reduces future maintenance costs, and ensures the software performs reliably under various conditions. It includes different levels such as unit, integration, system, and acceptance testing, all of which contribute to delivering a robust application.

2.Application Features (Fitness App)
The fitness application developed in my bachelor thesis is designed to help users maintain and improve their physical health through goal tracking, personalized workout recommendations, and user-specific dashboards. Key features include user registration and authentication, personalized onboarding (such as age setup), exercise plans, progress tracking, and motivational alerts. It also supports persistent login functionality through the “Remember Me” option and offers a smooth user experience with secure data handling and responsive UI across mobile devices using React Native.

3.Testing Techniques
During development, various testing techniques should be employed to ensure software quality. Black-box testing will be used to validate inputs and outputs without knowing internal code structures, especially for forms like login and registration. White-box testing will validate logic and decision paths, such as conditions for automatic login or password visibility toggling. Boundary value testing will be helpful in input validation (e.g., age, password length). Regression testing will be important whenever updates are made to ensure that existing features like login and user redirection continue functioning properly.

4.Unit Testing (Login Feature)
For unit testing, the handleLogin function in the login component is a key candidate. This function handles user authentication, checks input validity, performs API calls, and navigates users based on their profile state. Unit testing can involve mocking the API responses using a tool like Jest or React Testing Library, and validating different scenarios: missing inputs, invalid credentials, successful login, and redirection based on whether age is set. Tests should also verify behavior when the “Remember Me” checkbox is toggled, ensuring credentials are stored or cleared accordingly.

5.Integration Testing
Integration testing should focus on how modules like the login form, local storage (via AsyncStorage), and remote API services work together. Key integrations include: the form inputs and the login logic, the login logic and API endpoints, and finally the login result and the navigation router (useRouter). A suitable strategy is bottom-up integration testing, where we first ensure that individual modules like API calls and AsyncStorage handling are working, and then test their integration into the full login sequence. This ensures each dependency is functional before integrating them.

6.System Testing
System testing evaluates the complete fitness application as a whole. It should test end-to-end scenarios such as: new user registration, login with correct credentials, incorrect login attempts, auto-login using saved credentials, onboarding navigation when age is not set, and smooth transition to the home screen. Other system tests would include edge cases like input validation failures, network failures during API calls, and verifying UI responsiveness across device types. These tests ensure that all components interact seamlessly and meet functional and non-functional requirements.